Committed to Excellence

ALIGARH INSTITUTE OF TECHNOLOGY

Diploma Information Technology 2-Years (3rd Semester)

Stack Kart

Django E-Commerce Platform

Project Report

CERTIFICATE

This is to certify that the students of

D.I.T 2nd YEAR (3rd SEMESTER)

has carried out the necessary project titled

"STACK KART"

Submitted by

GROUP MEMBERS

AIMAN	**(E/012)**
SYED MUHAMMAD UMAR	**(E/008)**

Project Advisor

Head Of Department

TABLE OF CONTENTS

ACKNOWLEDGEMENT

First and foremost, I am grateful to my project supervisor and respected teacher for their continuous support, guidance, and valuable feedback throughout the development of this project. Their mentorship was instrumental in understanding the core concepts of full-stack web development with Django and applying them effectively in a real-world scenario.

I would also like to thank my institution for providing the necessary academic environment, resources, and encouragement that made this project possible.

Finally, I appreciate the support of my family and friends who stood by me during this journey. Thank you.

- StackKart Development Team

PROJECT DESCRIPTION

StackKart is a full-stack e-commerce web application built using Django 5 and Django REST Framework. It is designed as a professional academic/final-year project and features a dark and modern storefront, product catalog, shopping cart, checkout flow, order management, and Pakistan-ready payment options including Cash on Delivery, JazzCash, and EasyPaisa.

Key Functional Components:

- **Information Pages:** The site structure includes a product grid homepage, category filtering, product detail pages, shopping cart, checkout with delivery details, and order confirmation.
- **Dynamic Content:** The project features client-side interactivity including a JavaScript-powered live cart preview on the homepage that updates in real time when items are added or removed.
- **Cart Management System:** The cart system supports both anonymous users (via session keys) and logged-in users. Cart totals are calculated dynamically and displayed in the navbar.
- **Order & Payment Processing:** The checkout process collects shipping details and payment method selection. Orders are created atomically with stock deduction, and payment records are stored for COD, JazzCash, and EasyPaisa.

Technical and Design Scope:

The design uses a custom dark theme inspired by GitHub's color palette (#0d1117 background, #3fb950 green accent). It utilizes CSS Flexbox and Grid for responsive layouts, ensuring optimal display across all device types. The backend follows Django's best practices with ORM-based database access, CSRF protection, API throttling, and Argon2 password hashing.

OBJECTIVES

The development of StackKart was driven by several key objectives aimed at building a functional, user-centric, and technically sound e-commerce platform.

Functional E-Commerce Features:

The primary goal was to implement a complete online shopping experience. This includes a product catalog with search and category filtering, a shopping cart that works for both guests and registered users, and a checkout process that captures delivery details and processes orders with payment tracking.

User Engagement & Interactivity:

A key objective was to enhance user experience through dynamic design elements. This was achieved by integrating JavaScript for the live cart preview on the homepage and real-time cart count updates in the navigation bar, providing instant feedback to user actions.

Cross-Platform Responsiveness:

The project aimed to ensure optimal accessibility across devices of all sizes. The design uses modern CSS techniques including Flexbox and Grid to maintain visual appeal and usability on desktops, tablets, and mobile devices.

Admin Management Capabilities:

A fundamental objective was to provide administrators with a comprehensive management interface. Using Django's built-in admin panel, administrators can manage products, categories, customer profiles, view and manage orders, export data as CSV, and review payment records.

Secure Development Practices:

The project demonstrates adherence to Django security best practices including CSRF protection, ORM-based database access to prevent SQL injection, Argon2 password hashing, HTTP-only session cookies, API throttling, and environment-based configuration through .env files.

SYSTEM REQUIREMENTS

I. Software Requirements

Front-End (Client-Side):

- HTML5 - Structural markup for all Django templates (base.html, home.html, cart.html, etc.)
- CSS3 - All styling, presentation, and responsive design (stackkart.css)
- JavaScript - Client-side interactivity including live cart preview and cart management (stackkart.js)

Back-End (Server-Side):

- Python 3.11+ - Core programming language
- Django 5 - Full-stack web framework
- Django REST Framework - REST API for cart and products

Database Management System:

- SQLite - Default development database
- PostgreSQL - Recommended for production deployment

Additional Dependencies:

- django-environ - Environment variable management
- django-cors-headers - CORS support
- drf-spectacular - Swagger/OpenAPI documentation
- argon2-cffi - Secure password hashing
- Pillow - Image handling

II. Development Environment

- Code Editor: VS Code, PyCharm, or any Python-compatible IDE
- Web Browser: Chrome, Firefox, Edge (for cross-browser testing)
- Version Control: Git

PROJECT MANUAL

I. Navigation and Interface

- Global Navigation: The sticky navigation bar provides constant access to key pages: Products, Cart, Login/Register, and Admin.
- Footer Links: The footer includes quick links to Shop, Account, and Payments sections, ensuring easy access to all areas.
- Responsiveness: The layout dynamically adjusts to screen sizes. On mobile devices, the product grid converts to a single column for optimal usability.

II. Customer Features

- Browse & Filter Products: Users can browse products on the homepage grid, search by keyword, or filter by category using the pill-shaped category buttons.
- Add to Cart (AJAX): Clicking "Add to Cart" sends an AJAX request to the cart API, updating the navbar cart count and the live cart preview in real time.
- Live Cart Preview: The live cart preview on the homepage displays the latest added product, quantity, cart total, and item count - all updated dynamically.
- Manage Cart: On the cart page, users can update quantities or remove items. The order summary sidebar shows the subtotal and total.

III. Checkout & Order Processing

- Checkout Form: Users must be logged in to proceed to checkout. The checkout form collects full name, phone, city, and shipping address.
- Payment Selection: Users can select Cash on Delivery, JazzCash, or EasyPaisa as their payment method.
- Order Placement: Placing an order atomically creates the order, deducts stock, creates a payment record, and empties the cart.

IV. Admin Features

- Admin Panel: The Django admin panel (/admin/) allows management of products, categories, orders, carts, and payment records.
- Order Management: Administrators can filter orders by status, view order details, and export selected orders as CSV files.

WEBSITE STRUCTURE

I. File Organization (Modular Django App Structure)

The project follows Django's best practices with a modular structure separating concerns across multiple apps:

- accounts/ - User registration, login, logout, customer profiles
- catalog/ - Product and category models, homepage views, REST API
- cart/ - Cart and cart item models, checkout views, cart API endpoints
- orders/ - Order and order item models, order creation service
- payments/ - Payment records and gateway placeholder classes
- config/ - Django settings, URL configuration, ASGI/WSGI entry points
- static/ - CSS (stackkart.css), JavaScript (stackkart.js), images (favicon.svg)
- templates/ - HTML templates organized by app (accounts, cart, catalog)

II. Site Architecture (URL Routes)

The site follows a clean URL structure:

Route	Description
/	Homepage & product grid
/products/<slug>/	Product detail page
/cart/	Shopping cart page
/checkout/	Checkout & payment page
/order-success/<id>/	Order confirmation page
/accounts/login/	Customer login
/accounts/register/	Customer registration
/admin/	Django admin panel
/api/v1/products/	Products REST API
/api/v1/cart/	Cart REST API
/api/docs/	Swagger API documentation

SCREEN DESIGNS

I. Design Principles

Color Palette:

The design uses a dark theme inspired by developer-focused interfaces. The primary background is #0d1117 (deep navy), with card surfaces at #161b22 and borders at #30363d. The accent color is #3fb950 (green), creating high contrast for action buttons and interactive elements. Text is #e6edf3 for readability, with #8b949e for secondary/muted text.

Typography:

The design uses the Inter font family with system-ui fallbacks. Headings are bold with tight letter-spacing (-0.055em), creating a modern, developer-oriented aesthetic.

Layout:

The overall layout is responsive, with a three-column product grid on desktop that collapses to two columns on tablets and a single column on mobile. All content sections use CSS Grid and Flexbox for alignment.

II. Key Screen Layouts

1. Homepage (home.html):

Hero Section: Features the brand tagline, a search bar with keyword search, and a "Live Cart" preview card on the right. The cart preview shows the latest added product, quantity, category, current total, and a "View Cart & Checkout" button - all updated in real time via JavaScript.

Category Pills: Below the hero, a horizontal row of pill-shaped buttons for each category. Active pill is highlighted in green; clicking a pill filters the product grid.

Product Grid: A responsive grid of product cards. Each card shows the product image, name, price, an "Add to Cart" button, and a quick-view link.

2. Product Detail (product_detail.html):

Displays a large product image on the left and product details (name, price, description, stock status) on the right. Includes "Add to Cart" and a quantity selector.

3. Cart Page (cart.html):

Lists all items in the cart with quantity steppers and remove buttons. A sticky order summary sidebar on the right shows subtotal, total, and a "Checkout" button.

4. Checkout & Order Success:

The checkout page contains a delivery form (name, phone, address, city) and payment method selection (COD / JazzCash / EasyPaisa). The order success page shows a confirmation with order ID, items, and total.

5. Login / Register (login.html, register.html):

Centered card layouts on a dark background. Each form includes labeled input fields with placeholders, error messages, and a submit button. Links allow switching between login and registration.

DATABASE DESIGN

The StackKart project uses Django's ORM to manage 10 database tables. The database is SQLite for development and can be switched to PostgreSQL for production via environment variables.

I. Entity Relationship Overview

The database consists of the following main entities: User, CustomerProfile, Category, Product, Review, Cart, CartItem, Order, OrderItem, and PaymentRecord.

II. Main Tables and Schemas

User (Django Built-in):

Stores authentication data: username, email, password, first/last name, and account status.

CustomerProfile:

Extends User with: phone (varchar 20), address (text), city (varchar 80), and created_at (datetime).

Category:

Fields: name (varchar 120, unique), slug (varchar 140, unique), parent (self-referential FK). Supports nested categories.

Product:

Fields: category (FK), name (varchar 180), slug (varchar 220, unique), description (text), price (decimal 10,2), stock (positive int), image_url (URL), is_active (boolean), created_at, updated_at. Indexed on category, is_active, and slug.

Review:

Fields: product (FK), user (FK), rating (small int, default 5), body (text), created_at. Unique constraint on (product, user).

Cart:

Fields: user (FK, nullable for guest users), session_key (varchar 80), created_at, updated_at. Total is a calculated property from cart items.

CartItem:

Fields: cart (FK), product (FK), quantity (positive int). Unique on (cart, product). Line total is a calculated property.

Order:

Fields: user (FK), status (choice: pending/confirmed/shipped/delivered/cancelled), total (decimal 10,2), shipping_address (text), created_at, updated_at. Indexed on user and status.

OrderItem:

Fields: order (FK), product (FK), quantity (positive int), unit_price (decimal 10,2). Line total is a calculated property.

Payment Record:

Fields: order (FK), gateway (choice: jazzcash/easypaisa/cod), gateway_reference (varchar 120), amount (decimal 10,2), status (choice: initiated/success/failed), raw_response (JSON), created_at.

FLOW CHART

The following flow chart represents the complete user journey through the StackKart e-commerce platform, from browsing to order placement.

StackKart Shopping Flow:

- 1. Start: User visits StackKart homepage.
- 2. Browse: User views products on the homepage grid.
- 3. Search/Filter: User searches by keyword or filters by category.
- 4. View Product: User clicks on a product to view details.
- 5. Add to Cart: User clicks "Add to Cart" - AJAX POST to /api/v1/cart/add/.
- 6. Cart Updated: Navbar count and live cart preview update in real time.
- 7. View Cart: User navigates to /cart/ to review items.
- 8. Manage Cart: User can update quantities or remove items.

----- Decision Point: Is the user logged in? -----

- No -> Redirect to Login/Register page
- User logs in or creates an account
- Yes -> Proceed to Checkout
- 9. Checkout: User fills delivery details (name, phone, city, address).
- 10. Payment: User selects COD, JazzCash, or EasyPaisa.
- 11. Place Order: POST request processes the order.

----- Backend Process (Atomic Transaction) -----

- Create Order record
- Deduct product stock
- Create OrderItem records
- Create PaymentRecord
- Clear cart items
- 12. Success: User is redirected to order confirmation page.
- 13. End: User can continue shopping.

SCREENSHOTS OF THE WEBSITE

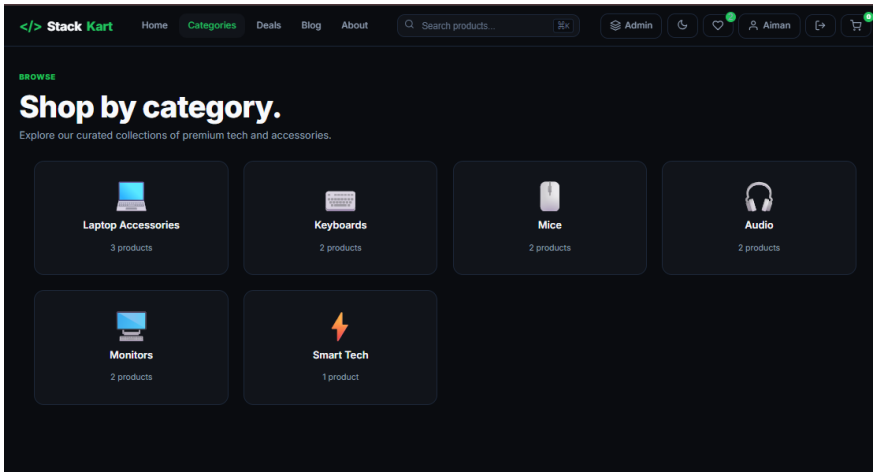
The following screenshots illustrate the main pages of the StackKart e-commerce platform - the customer-facing storefront, the checkout flow, the authentication pages, and the custom-themed Django admin panel. Drop the matching PNG files into the screenshots/ folder at the project root using the filenames referenced below.

1. Homepage:

Main landing page of StackKart with a dark theme, blended hero image, filter sidebar, category pills, sort bar, and responsive product grid.

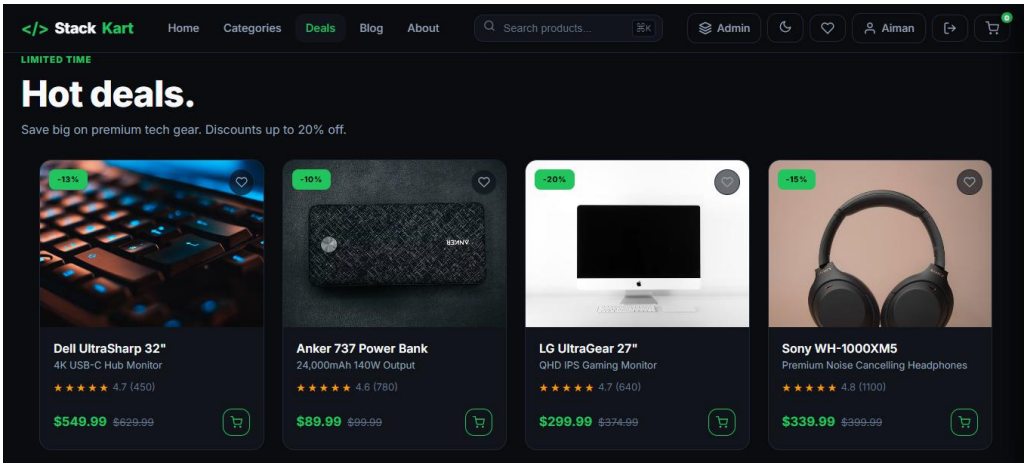
2. Categories Page:

Browse-all-categories view - every product category rendered as a card with icon, name, and product count.



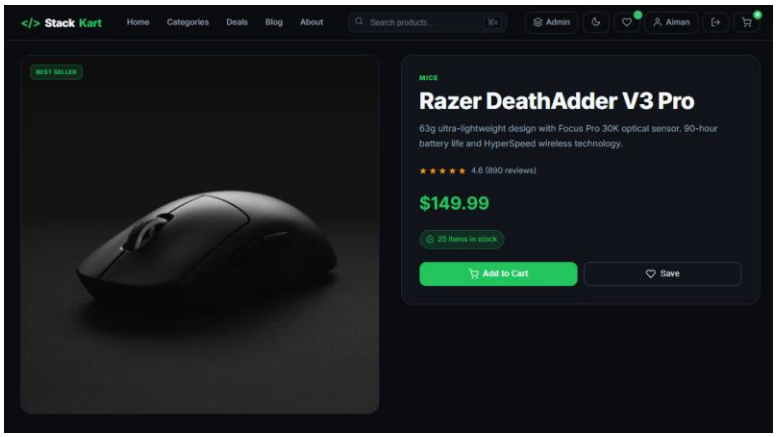
3. Deals Page*:*

Hot deals listing - discounted products showing strike-through original price, deal price, and a 'DEAL' tag.

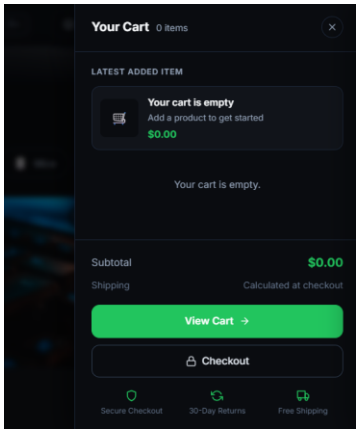


4. Product Detail Page*:*

Single product view with image, title, subtitle, price, rating, description, Add to Cart, wishlist heart, Notify Me stock alert, and a 'Complete Your Setup' bundle section.

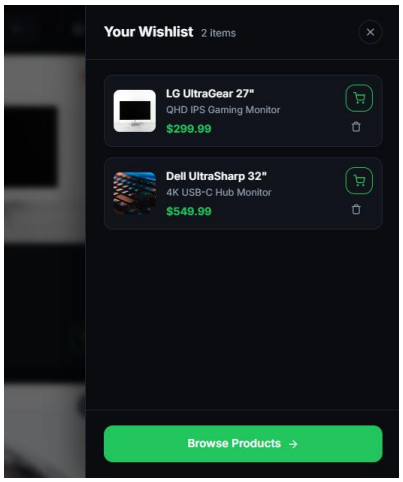


5. Cart Sidebar*:* Right-side slide-out cart drawer with line items, quantity steppers, live subtotal, and a green 'Checkout' button.



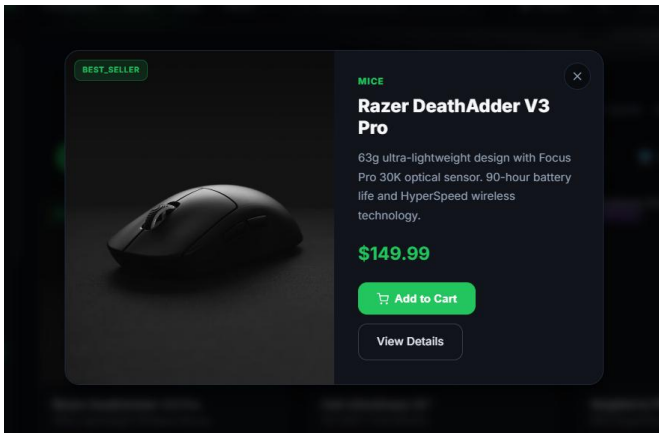
6. Wishlist Panel*:*

Right-side wishlist drawer showing saved products with a remove button and a 'synced to your account' indicator.



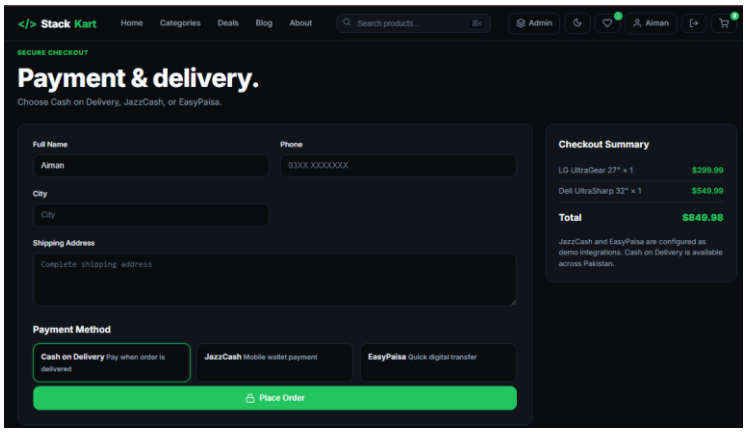
7. Quick-View Modal*:*

Modal popup that opens from any 'Quick View' trigger on a product card - image, title, price, and Add to Cart without leaving the current page.



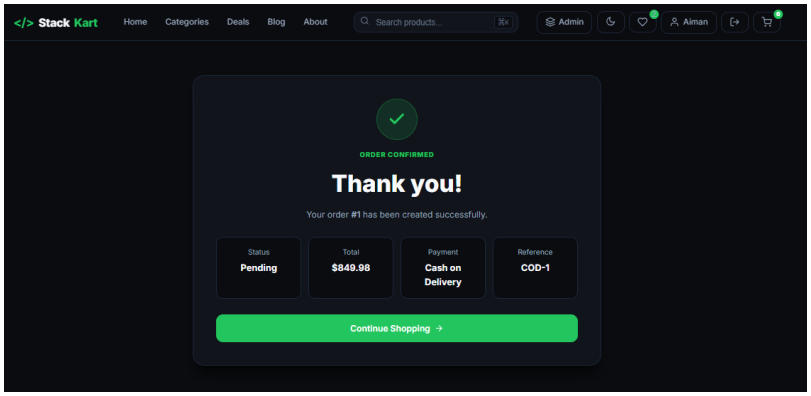
8. Checkout*:*

Multi-step checkout with delivery form (name, phone, address, city), payment method radio buttons (Cash on Delivery / JazzCash / EasyPaisa), and an order summary on the right.



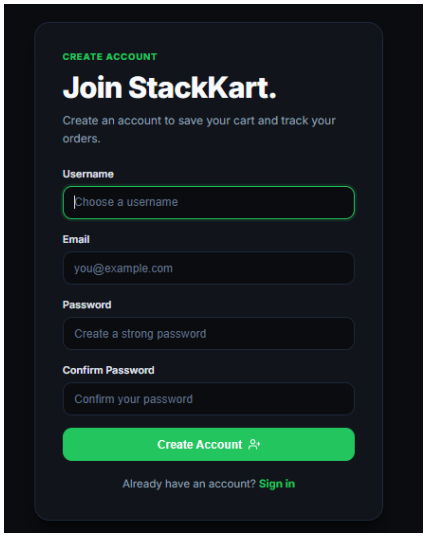
9. Order Success:

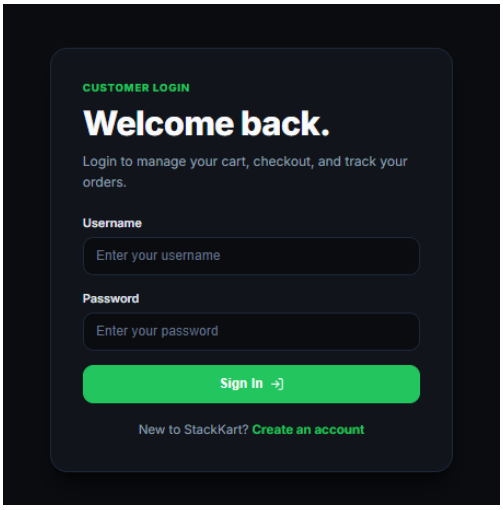
Confirmation page after placing an order - order ID, status badge, item list, total amount, and a 'Continue Shopping' button.



10. Login / Register:

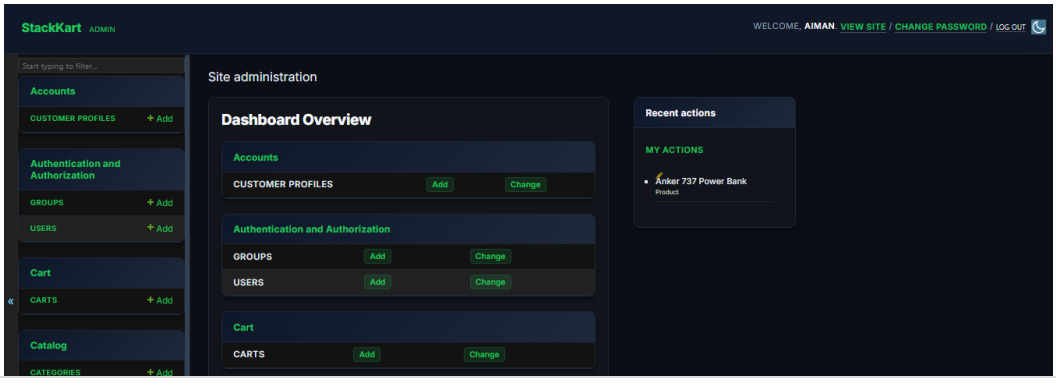
Auth pages with the same dark StackKart theme - clean centered card with email, password fields, and a primary green submit button.





11. Admin Dashboard:

Custom Django admin home - StackKart branding in the header (no 'Django administration' title), dark background, card-style app modules, recent actions sidebar, and green accent colors throughout.



DJANGO BACKEND IMPLEMENTATION

I. Settings Configuration (config/settings.py)

The settings module uses django-environ for environment variable management. Key settings include SECRET_KEY, DEBUG, ALLOWED_HOSTS, and DATABASE_URL. The project is configured with Argon2 password hashing, DRF with throttling (20/min anonymous, 100/min authenticated), drf-spectacular for API documentation, CORS headers, and HTTP-only session cookies.

II. URL Configuration (config/urls.py)

The root URL configuration includes Django admin, accounts (auth), cart (checkout pages), catalog (product pages), REST API endpoints, and Swagger API documentation.

III. Cart API Endpoints (cart/api_urls.py & cart/views.py)

```
from django.urls import path
from . import views

urlpatterns = [
    path('cart/', views.cart_detail, name='cart-detail'),
    path('cart/add/', views.add_to_cart, name='cart-add'),
    path('cart/item/<int:item_id>/', views.cart_item_detail, name='cart-
item-detail'),
]
```

IV. Order Creation Service (orders/services.py)

The create_order_from_cart function uses @transaction.atomic to ensure all database operations succeed or fail together. It creates an Order record, deducts product stock, creates OrderItem records, and clears the cart - all within a single database transaction.

V. Payment Gateways (payments/gateways.py)

```
class JazzCashGateway:
    def initiate_payment(self, order):
        return {'status': 'initiated', 'reference': f'JC-{order.id}'}

class EasyPaisaGateway:
    def initiate_payment(self, order):
        return {'status': 'initiated', 'reference': f'EP-{order.id}'}
```

VI. Product & Category API (catalog/api_urls.py & catalog/views.py)

Django REST Framework ViewSets provide read-only access to categories and full CRUD for products. Search and ordering filters are configured for name, description, category, price, and creation date.

TESTING & VALIDATION

Testing and validation were conducted to ensure the website meets the defined objectives and performs reliably across all features.

I. Functional Testing

Cart API Test (Add to Cart):

A POST request to /api/v1/cart/add/ with a valid product_id and quantity was tested. Result: The item was added to the cart, the response returned the updated cart with correct item count and total. Stock validation also correctly rejected requests exceeding available stock.

Cart Update & Delete Test:

PATCH requests to /api/v1/cart/item/<id>/ with updated quantities and DELETE requests to remove items were validated. Result: Quantities updated correctly, items removed, and cart total recalculated.

Checkout & Order Flow Test:

A complete checkout flow was simulated with a logged-in user having items in the cart. Delivery details were submitted with a selected payment method. Result: Order created successfully, stock deducted, payment record created with correct gateway reference, cart emptied, and user redirected to order success page.

Admin CSV Export Test:

The admin action "Export selected orders as CSV" was tested on a selection of orders. Result: A valid CSV file was generated with correct headers (ID, User, Status, Total, Created) and the selected order data.

II. Non-Functional Testing

Responsiveness Test:

The website's layout was tested across different screen sizes. Result: The product grid correctly converts from 3 columns (desktop) to 2 columns (tablet) to 1 column (mobile). All navigation elements remain accessible.

API Throttling Test:

Rapid repeated requests to the cart API were made from an anonymous session. Result: After exceeding 20 requests per minute, the API correctly returned 429 Too Many Requests.

Security Validation:

CSRF protection was verified by submitting POST requests without a valid CSRF token. Result: Django correctly rejected these requests with 403 Forbidden. Password hashing was verified by checking that stored passwords use the Argon2 hasher.

CONCLUSION

The development of Stack Kart successfully achieved all stated project objectives, resulting in a fully functional, responsive, and complete e-commerce platform built with Django. The project demonstrates comprehensive understanding of full-stack web development principles, integrating modern front-end technologies (HTML5, CSS3, JavaScript) with a robust Django backend and REST API architecture.

Key successes include:

- Complete E-Commerce Flow: The implementation of product browsing, cart management, checkout, and order processing provides a seamless shopping experience from start to finish.
- Pakistan-Ready Payments: The integration of Cash on Delivery, JazzCash, and EasyPaisa payment options with PKR currency makes the platform immediately relevant for the local market.
- Dynamic User Experience: The JavaScript-powered live cart preview and real-time cart count updates provide instant feedback and a modern, interactive feel.
- Admin Capabilities: The comprehensive admin panel with product management, order tracking, and CSV export empowers administrators to manage the store efficiently.
- Technical Integrity: Django security best practices - CSRF protection, ORM-based queries, Argon2 hashing, API throttling, and environment-based configuration - ensure a secure and maintainable application.

The project successfully fulfills the requirements of this academic submission and establishes a solid foundation for future expansion. Potential enhancements could include real JazzCash/EasyPaisa API integration, customer dashboard, wishlist feature, coupon/discount system, invoice PDF generation, and admin analytics dashboard.

- End of Report -